

dept-track-multi, Explained

How a multi-company Dependency-Track vulnerability platform is built, piece by piece — and how the three companies are combined into one view.

1. The big picture: what we built and why

The goal of **dept-track-multi** is to simulate an organisation made of three separate companies (or departments) — Acme, Beta and Gamma — where each one runs its own independent software-vulnerability dashboard. For every company we automatically produce a list of the open-source libraries its sample applications depend on, feed that list into a security tool that checks each library against known-vulnerability databases, and finally pull all three companies' results together into a single web page so you can see the whole portfolio at a glance.

Everything runs on one Linux host using Docker, so the entire system — three full vulnerability platforms, the data, and the supporting scripts — can be started with one command and torn down just as easily. Keeping each company on its own instance (rather than mixing them in one) mirrors the real world, where different business units often need their data and access kept separate.

2. Background concepts (so the rest makes sense)

SBOM — Software Bill of Materials

An SBOM is an ingredient list for software. Modern applications are built from hundreds of open-source packages, and each of those can contain security flaws. An SBOM enumerates every component (name, version, and a unique identifier) so tools can reason about what is actually inside an application.

CycloneDX — the SBOM format we use

CycloneDX is a widely-adopted standard file format for SBOMs, usually written as JSON. Like any standard it has version numbers (1.4, 1.5, 1.6, 1.7...). This matters a great deal here: a tool that writes a newer version than the consumer understands will be rejected. We hit exactly this — the first generator emitted version 1.7, which our platform could not read — and the fix was to pin the format to 1.5.

Dependency-Track — the platform

Dependency-Track (DT for short) is an open-source platform that continuously analyses SBOMs. You upload an SBOM for a project, and DT matches every component against vulnerability feeds such as the National Vulnerability Database (NVD), the GitHub Advisory Database and OSV, then shows you the risks with severities and a risk score. Each DT instance is made of two parts: an **API server** (the brain — a Java application that stores data and runs the analysis) and a **frontend** (the web user interface you open in a browser).

Syft — the SBOM generator

Syft is a small, fast tool that scans a project folder and produces an SBOM. It is a 'cataloguer': it reads the package files that are already present (lock files, installed packages) and lists them. We chose it over the heavier cdxgen because it is far smaller (about 90 MB versus 16 GB) and, once pinned to CycloneDX 1.5, its output uploads cleanly into Dependency-Track.

3. Architecture and topology

Three companies, each with its own API server plus frontend, all on one host. Each instance keeps its data in its own Docker volume, so the three are fully isolated. The table below is the live layout (note that Acme's API was later moved from port 8081 to 18081 to dodge a port clash):

Dept	Company	API port	UI port	Projects	Stack
acme	Acme Corp	18081	8091	express, juice-shop	Node.js
beta	Beta Industries	8082	8092	fastapi, flask	Python
gamma	Gamma Solutions	8083	8093	guava, spring-petclinic	Java

A request flows like this: your browser opens a company's **frontend** (for example port 8091 for Acme); the frontend then makes calls to that company's **API server** (port 18081). Because the calls are made by the browser, the API address the frontend advertises must be one your browser can actually reach — a detail that drives the whole networking section later.

4. The components, one by one

4.1 config.py — the single source of truth

Everything in the system is driven by one small Python file, config.py. It defines a dictionary called COMPANIES, where each entry holds the company's display name, the URLs of its API and frontend, the admin password to set, and the list of projects to create. A tiny helper function builds each entry so the URLs are assembled from a single SERVER_HOST value. Because every other script imports this file, changing the topology — adding a company, renaming a project, moving a port — means editing only this one place.

```
SERVER_HOST = os.environ.get("SERVER_HOST", "127.0.0.1")
COMPANIES = {
    "acme": _company("Acme Corp", 18081, 8091, [
        {"name": "express", "version": "latest", "description": "..."},
        {"name": "juice-shop", "version": "latest", "description": "..."}]),
    ... beta, gamma ...
}
```

4.2 docker-compose.yml — defining the containers

Docker Compose is a way to describe a set of containers in one file and start them together. Our file declares six services: an API server and a frontend for each of the three companies. Rather than copy the same settings six times, it uses YAML 'anchors' (a reusable block marked with &) and 'merge keys' (<<) so each service inherits a common template and only overrides what is unique —

its ports and its data volume.

Each container listens on port 8080 internally; we 'publish' that to a different port on the host so the three instances do not collide (8091, 8092, 8093 for the UIs, and so on). The API servers each mount a named volume at /data, which is where Dependency-Track keeps its embedded database — that is what makes the data survive restarts. Two environment settings are important: EXTRA_JAVA_OPTIONS caps the Java memory (-Xmx2g) so three heavy instances can fit on a modest host, and API_BASE_URL tells each frontend where its API server lives. The \${SERVER_HOST} placeholder is filled in from the environment when the stack starts.

```
x-apiserver: &apiserver
  image: dependencytrack/apiserver:latest
  environment: { EXTRA_JAVA_OPTIONS: "-Xmx2g" }

services:
  acme-apiserver: { <<: *apiserver, ports: ["18081:8080"], volumes: ["acme-data:/data"] }
  acme-frontend: { <<: *frontend, ports: ["8091:8080"],
    environment: { API_BASE_URL: "http://${SERVER_HOST}:18081" } }
```

4.3 gen_sboms.sh — making the SBOMs with Syft

This shell script produces one SBOM per project. It first clones the six sample open-source projects (Express and Juice Shop for Acme, FastAPI and Flask for Beta, Guava and Spring PetClinic for Gamma) into a local folder. Then, for each project, it runs Syft inside Docker and writes a CycloneDX file named in the pattern **<dept>-<project>.cdx.json** — for example acme-express.cdx.json. That naming is a contract: the upload script later looks for files with exactly these names.

The crucial flag is the output format, cyclonedx-json@1.5, which pins the SBOM to CycloneDX version 1.5. This is the change that fixed the upload failure: the previous tool, cdxgen, defaulted to version 1.7 that Dependency-Track refuses. Syft is also a 'cataloguer', meaning it reports what is declared or installed; for the Java projects, which are not pre-built, it sees fewer components than a full dependency resolver would — acceptable for a learning demo.

```
docker run --rm -v "$SRC/$2":/src -v "$OUT":/out \
  anchore/syft:latest dir:/src -o "cyclonedx-json@1.5=/out/$1-$2.cdx.json"
```

4.4 setup_and_upload.py — talking to the Dependency-Track API

This is the script that brings each empty Dependency-Track instance to life. For every company it performs the same sequence, all over Dependency-Track's REST API.

- **Wait:** it polls the /api/version endpoint until the API server has finished booting.
- **Log in:** Dependency-Track ships with the account admin/admin and forces a password change on first use. The script logs in, performs that forced change to the password from config.py, and receives a session token.
- **Permissions and key:** Dependency-Track creates a built-in team called 'Automation'. The script grants that team all permissions and then mints a fresh API key for it — this key is what lets the rest of the automation act without a human login.

- **Create projects:** for each project in config.py it creates the project (skipping it if it already exists, so re-runs are safe).
- **Upload:** it reads the matching .cdx.json file, encodes it in base64, and PUTs it to the /bom endpoint. Dependency-Track then processes and analyses it in the background.

Finally the script saves every company's API key into api_keys.json. There are two kinds of credentials in play: a short-lived **bearer token** from the admin login (used to set up the team), and the long-lived **API key** (sent as the X-API-Key header) used for creating projects and uploading. The api_keys.json file is the bridge that the reporting script reuses later.

4.5 run.sh — the conductor

run.sh ties everything together so the whole system comes up with one command. It first works out SERVER_HOST (from the environment, or by guessing the host's IP) and the admin password, then checks that Docker, Python and git are present. It then runs the steps in a deliberate order: start the containers first (so they begin their slow boot), generate the SBOMs while that boot happens, and only then run the upload script, which waits for each API to be ready before pushing data. Two switches, SKIP_UP and SKIP_SBOMS, let you re-run just part of the pipeline. At the end it prints the dashboard URLs.

```
SERVER_HOST=192.168.112.134 ./run.sh      # full run
SERVER_HOST=192.168.112.134 SKIP_SBOMS=1 ./run.sh  # reuse existing SBOMS
```

4.6 combined_report.py — one view across three instances

Because the three companies live in three separate Dependency-Track databases, Dependency-Track itself cannot show them on one screen. This script fills that gap. It reads config.py to know the instances and api_keys.json to authenticate, then for each company and each project it asks the API three questions: what projects exist, what are the current metrics (component count, vulnerability counts by severity, risk score), and what are the individual findings. It sorts the findings so the most severe appear first and keeps the top few per project.

It then writes two files: a machine-readable combined JSON, and a single self-contained HTML dashboard with summary cards (totals across all companies) followed by a section per company and a card per project. Run with the --serve switch, it also starts a tiny web server so you can open the dashboard from your browser at http://<server-ip>:8090/combined_report_latest.html.

5. Why Dependency-Track makes this kind of custom implementation possible

None of the automation above is a clever hack against a closed product. Dependency-Track is built API-first: the web interface you click is simply one client of a complete REST API, and anything the UI can do, a script can do too. The specific design choices below are what let us drive three full instances entirely from a few short Python and shell files.

5.1 A documented REST API for everything

Every action we performed — logging in, the forced password change, listing and permissioning teams, minting an API key, creating projects, uploading SBOMs, reading metrics and findings — is an ordinary HTTP endpoint under `/api/v1`, described by an OpenAPI (Swagger) specification the server publishes. Because the API is the real interface and the UI merely sits on top of it, our scripts needed nothing more than Python's built-in HTTP client; there is no hidden, UI-only path.

5.2 Teams, permissions and API keys built for automation

Dependency-Track has a first-class access model: 'teams' hold granular permissions, and a team can own one or more API keys for machine-to-machine use. It even ships a default team literally called 'Automation' for exactly this purpose. We granted that team the permissions we needed (such as project creation and BOM upload) and generated a key, which we then sent in the `X-Api-Key` header on every request. This is why the automation can act safely without anyone's personal login, and with precisely the rights it requires and no more.

5.3 Container-native, configured entirely by environment

The platform is shipped as official Docker images and configured through environment variables rather than an interactive installer. State lives in a single `/data` volume (an embedded database by default, or an external PostgreSQL for production), and behaviour such as the frontend's API address or the Java heap size is set with a variable. That design is exactly what let us declare three independent instances in one Compose file and bring them up reproducibly, differing only by their ports and volumes.

5.4 Standards-based, asynchronous SBOM ingestion

Dependency-Track ingests the CycloneDX standard directly, so it does not care which tool produced the SBOM — Syft, Trivy or cdxgen all work, provided the format version is one it supports. Uploads go to a single `/bom` endpoint (we send the file base64-encoded inside JSON), and the server processes them in the background, creating components and running analysis after the request returns. Building on a published standard rather than a proprietary format is what makes the generator interchangeable.

5.5 Metrics and findings you can query

Crucially, Dependency-Track does the hard security work and then hands the results back. It mirrors vulnerability sources (NVD, the GitHub Advisory Database, OSV and others) internally, continuously re-checks components, and publishes per-project metrics and findings through the API. Our combined report computes no risk of its own — it simply asks each instance for its already-calculated metrics and findings and arranges them on one page. The platform doing the analysis and offering it back over HTTP is what made the cross-company view easy to assemble.

5.6 One instance or many — the topology is your choice

Finally, Dependency-Track is happy hosting many projects and teams inside a single instance, using its portfolio, tags and access control to keep them apart — which is how most organisations actually run it. Our decision to run three separate instances is a deliberate isolation choice for this demo, not a platform limitation. The same building blocks — an open API, automation-friendly access control, and

environment-driven containers — let you compose whichever shape you need. Beyond what we used, that same API-first design also exposes a policy engine, outbound notifications (webhooks, Slack, email) and VEX handling, so the platform can act as a composable backend inside a larger pipeline rather than a closed application. That composability is the real reason a custom build like this is straightforward.

6. How the pieces connect (end-to-end data flow)

Read top to bottom, the system is a chain where `config.py` is the source of truth and `api_keys.json` is the bridge from setup to reporting:

- You set `SERVER_HOST` and run `run.sh`.
- Docker Compose starts six containers; each frontend is told its API address via `API_BASE_URL`.
- Syft scans the six cloned projects and writes the `.cdx.json` SBOM files.
- `setup_and_upload.py` reads `config.py` and those SBOM files, authenticates to each instance, creates projects, uploads the SBOMs, and saves `api_keys.json`.
- Dependency-Track analyses each SBOM against vulnerability feeds and computes metrics.
- `combined_report.py` reads `config.py` and `api_keys.json`, queries every instance, and renders the combined dashboard, which it can serve over HTTP for your browser.

7. Networking: why `SERVER_HOST` and the ports matter

Inside its container each service listens on port 8080; Docker 'publishes' that to a unique port on the host. The subtle part is `API_BASE_URL`. The frontend is just static files sent to your browser, and it is your **browser** that then calls the API. So the API address must be reachable from wherever you are browsing — using 'localhost' only works if you browse from the server itself. That is why we set `SERVER_HOST` to the address your browser will use.

On this project the platform runs in a VMware virtual machine using NAT networking. A helpful property of VMware's NAT is that the host computer shares the same private network as the guest, so the host can reach the VM's IP (192.168.112.134) directly — no port forwarding and no administrator rights required. We also moved Acme's API from port 8081 to 18081 because a leftover container from an earlier run was still holding 8081; remapping the port sidestepped the conflict cleanly.

8. Putting it on GitHub (how we shipped it)

The deployment bundle was turned into a public GitHub repository so it can be pulled onto any server. Beyond the obvious commit-and-push, three details make it robust. A **.gitignore** keeps runtime secrets and bulk data (`api_keys.json`, the SBOMs, the cloned source) out of the repository. A **.gitattributes** forces Unix (LF) line endings, because shell scripts authored on Windows would otherwise carry carriage returns that make Linux refuse to run them. And the executable bit is recorded in git, so the scripts are runnable immediately after cloning. The result is a one-line clone-and-run on a fresh server.

```
git clone https://github.com/Jeeva-uva/dpt.git
cd dpt
SERVER_HOST=<server-ip> ./run.sh
```

9. Key decisions and lessons

- **Syft instead of cdxgen:** cdxgen emitted CycloneDX 1.7 (rejected by Dependency-Track) and its image is ~16 GB; Syft pinned to 1.5 is small and uploads cleanly.
- **Memory reality:** each API server is a memory-hungry Java process (~2–4 GB). Three of them need roughly 16 GB, which is why an 8 GB VM struggles — the levers are the heap size (-Xmx) and the number of instances, not the choice of container image.
- **Bundled vs split images:** Dependency-Track offers an all-in-one 'bundled' image and a split apiserver+frontend pair. We use the split pair; either way the cost is the Java API server.
- **Port remap over force-removal:** when a stuck container held port 8081 and could not be removed, moving the new service to 18081 was the pragmatic unblock.

10. Quick command reference

```
# bring the whole system up
SERVER_HOST=192.168.112.134 ./run.sh

# open a single company UI (admin / your password)
http://192.168.112.134:8091 # acme (also 8092 beta, 8093 gamma)

# build and serve the combined cross-company dashboard
python3 combined_report.py --serve
http://192.168.112.134:8090/combined_report_latest.html

# stop everything (keep data) / wipe data
docker compose down
docker compose down -v
```

That is the entire system: a single configuration file drives six containers; Syft turns source code into SBOMs; an upload script teaches each Dependency-Track instance about its projects; and a reporting script stitches the three separate instances back into one portfolio view.

Appendix A — Code walkthrough, file by file

This appendix goes through every file in the bundle and explains the code itself. Each monospace block is followed by a plain-language explanation of what it does and why. Some long files are shown as their essential pieces rather than every line.

A.1 config.py

```
SERVER_HOST = os.environ.get("SERVER_HOST", "127.0.0.1")
ADMIN_PASSWORD = os.environ.get("DT_ADMIN_PASSWORD", "ChangeMe123!")
```

These two lines read settings from the environment, each with a fallback default. Reading from the environment is what lets the very same file work on any machine: you pass SERVER_HOST and the admin password at run time instead of editing the file.

```
def _company(name, api_port, fe_port, projects):
    return {
        "name": name,
        "api_url": f"http://{SERVER_HOST}:{api_port}",
        "frontend_url": f"http://{SERVER_HOST}:{fe_port}",
        "default_password": "admin",          # DT factory default
        "new_password": ADMIN_PASSWORD,      # what we change it to
        "projects": projects,
    }
```

A small factory that assembles one company's record. It builds the API and frontend URLs from the single SERVER_HOST value plus the given port, records Dependency-Track's factory password ('admin') and the password we intend to switch to, and stores the project list. Centralising URL construction here means one SERVER_HOST change correctly updates every instance.

```
COMPANIES = {
    "acme": _company("Acme Corp", 18081, 8091, [
        {"name": "express", "version": "latest", "description": "..."},
        {"name": "juice-shop", "version": "latest", "description": "..."}]),
    "beta": _company("Beta Industries", 8082, 8092, [ ... ]),
    "gamma": _company("Gamma Solutions", 8083, 8093, [ ... ]),
}
```

The dictionary every other script imports. Each key is a short department id ('acme', 'beta', 'gamma'); each value is a company record. That key is reused as the SBOM filename prefix and as the key in api_keys.json, which is how the separate scripts keep their data lined up.

A.2 docker-compose.yml

```
x-apiserver: &apiserver
  image: dependencytrack/apiserver:latest
  environment: { EXTRA_JAVA_OPTIONS: "-Xmx2g" }
  restart: unless-stopped
  deploy: { resources: { limits: { memory: 3g } } }

x-frontend: &frontend
  image: dependencytrack/frontend:latest
  restart: unless-stopped
```


Keys beginning with x- are 'extension fields' that Compose ignores as services; the `&apiserver` and `&frontend` labels turn them into reusable anchors. They capture the shared template for each kind of container: which image to run, a capped Java heap, automatic restart, and a memory limit.

```
services:
  acme-apiserver: { <<: *apiserver, ports: ["18081:8080"],
                    volumes: ["acme-data:/data"] }
  acme-frontend: { <<: *frontend, ports: ["8091:8080"],
                  environment: { API_BASE_URL: "http://${SERVER_HOST}:18081" } }
```

Each service merges the template in with '`<<: *anchor`', then adds only what is unique. The `apiserver` adds its host-to-container port mapping (host 18081 to the container's 8080) and its data volume. The `frontend` adds `API_BASE_URL` — the browser-reachable address of its API server — where `${SERVER_HOST}` is filled in from the environment when the stack starts. `Beta` and `gamma` are the same with their own ports.

```
volumes:
  acme-data:
  beta-data:
  gamma-data:
```

Declares the three named volumes Docker manages on disk. Each is the home of one instance's embedded database, which is exactly what survives 'docker compose down' but is erased by 'down -v'.

A.3 gen_sboms.sh

```
set -euo pipefail
HERE="$(cd "$(dirname "$0")" && pwd)"
SRC="$HERE/src"; OUT="$HERE/sboms"
mkdir -p "$SRC" "$OUT"
```

'set -euo pipefail' makes the script strict — it stops on the first error, on any unset variable, and on a failure anywhere in a pipe. `HERE` resolves the script's own directory so the paths work no matter where you launch it from, and the script then ensures the source and output folders exist.

```
SYFT=anchore/syft:latest
docker pull "$SYFT"
clone() { [ -d "$SRC/$2/.git" ] || git clone --depth 1 "$1" "$SRC/$2"; }
```

It pulls the Syft image once. `clone()` is a helper: if the target folder is not already a git checkout it shallow-clone (`--depth 1`, just the latest commit — fast and small) the given URL into it; otherwise it does nothing, so re-running the script does not re-download everything.

```
gen() {
  docker run --rm -v "$SRC/$2":/src -v "$OUT":/out \
    "$SYFT" dir:/src -o "cyclonedx-json@1.5=/out/$1-$2.cdx.json" \
    || echo "WARNING: syft had issues for $1-$2"
}
```

`gen <dept> <project>` runs Syft in a throwaway container (`--rm`). It mounts the project source at `/src` and the output folder at `/out`; `'dir:/src'` tells Syft to scan that directory; and `'-o cyclonedx-json@1.5=...'` writes a CycloneDX 1.5 file to the exact name the upload script expects. The trailing `'|| echo'` means

one troublesome project logs a warning instead of aborting the run.

```
clone https://github.com/expressjs/express.git express
gen acme express
# ... and the same pattern for the other five projects
```

The rest of the script is just calls to clone each demo repository and then gen with its department and project name, producing the six SBOM files.

A.4 setup_and_upload.py

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, BASE_DIR)
from config import COMPANIES
SBOM_DIR = os.path.join(BASE_DIR, "sboms")
KEYS_FILE = os.path.join(BASE_DIR, "api_keys.json")
MAX_WAIT = 600
```

It works out its own folder, adds it to the import path so 'from config import COMPANIES' works, and derives the SBOM folder and the key-file path relative to itself (which is what makes the bundle portable). MAX_WAIT is how many seconds it will wait for each API to come up.

```
def api_request(url, method="GET", data=None, headers=None, timeout=30):
    if isinstance(data, (dict, list)):
        data = json.dumps(data).encode("utf-8")
        headers.setdefault("Content-Type", "application/json")
    req = urllib.request.Request(url, data=data, headers=headers, method=method)
    try:
        with urllib.request.urlopen(req, timeout=timeout) as resp:
            return resp.status, resp.read().decode("utf-8")
    except urllib.error.HTTPError as e:
        return e.code, (e.read().decode() if e.fp else "")
```

A thin wrapper around Python's built-in HTTP client used for every JSON call. It serialises a dict or list body to JSON, sends the request, and — importantly — catches HTTP errors so a 4xx/5xx comes back as a normal (status, body) pair instead of raising. That keeps all the calling code simple.

```
def wait_for_api(api_url, label):
    start = time.time()
    while time.time() - start < MAX_WAIT:
        status, _ = api_request(f"{api_url}/api/version", timeout=5)
        if status == 200:
            return True
        time.sleep(5)
    return False
```

Polls the harmless /api/version endpoint every five seconds until it answers 200 (the server is up) or the timeout passes. This is why the upload step can be started immediately after the containers launch — it simply waits for each one to be ready.

```
def login(api_url, username, password):
    # POST form-encoded username + password to /api/v1/user/login
    # returns: the bearer token, or "FORCE_PASSWORD_CHANGE", or None

def force_change_password(api_url, old_pw, new_pw):
    # POST to /api/v1/user/forceChangePassword (one-time, first login)
```

Dependency-Track's login takes form-encoded credentials (not JSON) and returns a JWT bearer token on success. A brand-new instance forces the admin account to change its password first and signals this with the marker `FORCE_PASSWORD_CHANGE`; the code detects that, calls `forceChangePassword` to set our password, and logs in again. The main loop tries the new password first (for re-runs) and falls back to the default 'admin'.

```
def setup_automation_team(api_url, token):
    teams = ... # GET /api/v1/team (Bearer token)
    auto = next(t for t in teams if t["name"] == "Automation")
    for perm in ALL_PERMISSIONS: # 14 permissions
        # POST /api/v1/permission/{perm}/team/{auto.uuid}
        ...
    # PUT /api/v1/team/{auto.uuid}/key -> a fresh API key
```

Using the admin bearer token, it finds the built-in 'Automation' team, grants it any of the 14 permissions it is missing (BOM upload, project creation, view vulnerability, and so on), then generates a brand-new API key — existing keys are shown masked and cannot be reused. That returned key is the credential every later request uses.

```
def create_project(api_url, api_key, name, version, description):
    # GET projects; if one matches name+version, reuse its uuid
    # otherwise PUT /api/v1/project and return the new uuid

def upload_sbom(api_url, api_key, project_uuid, sbom_path):
    sbom_b64 = base64.b64encode(open(sbom_path, "rb").read()).decode()
    data = {"project": project_uuid, "bom": sbom_b64}
    # PUT /api/v1/bom with header X-API-Key
```

`create_project` is idempotent: it reuses an existing project of the same name and version if there is one, otherwise it creates the project and returns its UUID — so re-runs are safe. `upload_sbom` reads the SBOM file, base64-encodes it (DT's JSON BOM endpoint expects the document as a base64 string), and PUTs it tied to that UUID. Notice authentication has switched from the bearer token to the X-API-Key header here.

```
for company_id, company in COMPANIES.items():
    wait_for_api(...)
    token = login(...) # plus the forced password change
    api_key = setup_automation_team(...)
    api_keys[company_id] = api_key
    for proj in company["projects"]:
        uuid = create_project(...)
        upload_sbom(..., f"{company_id}-{proj['name']}.cdx.json")
json.dump(api_keys, open(KEYS_FILE, "w"))
```

The main loop ties it together: for each company it waits, authenticates, sets up the API key, then for each project creates it and uploads the matching SBOM — note the filename '`<dept>-<project>.cdx.json`', the very pattern `gen_sboms.sh` wrote. At the end it saves all the keys to `api_keys.json` for the reporting step to reuse.

A.5 run.sh

```
set -euo pipefail
cd "$(dirname "$0")"
if [ -z "${SERVER_HOST:-}" ]; then
```

```

SERVER_HOST="$(hostname -I | awk '{print $1}')"
echo "WARNING: SERVER_HOST not set; guessing $SERVER_HOST"
fi
export SERVER_HOST
export DT_ADMIN_PASSWORD="${DT_ADMIN_PASSWORD:-ChangeMe123!}"

```

It moves into its own directory, then resolves `SERVER_HOST` — if you did not set it, it guesses the host's first IP and warns you. It exports `SERVER_HOST` so Docker Compose can substitute `$(SERVER_HOST)`, and exports the admin password so `config.py` and the upload script agree on the same value.

```

command -v docker >/dev/null 2>&1 || { echo "ERROR: docker"; exit 1; }
docker compose version >/dev/null 2>&1 || { echo "ERROR: compose"; exit 1; }

```

Prerequisite checks: it confirms docker, the compose plugin, python3 and git are all present and fails early with a clear message if any is missing, rather than half-running and breaking later.

```

[ "$SKIP_UP" != "1" ]    && docker compose pull && docker compose up -d
[ "$SKIP_SBOMS" != "1" ] && bash gen_sboms.sh
python3 setup_and_upload.py
# finally: print the dashboard URLs

```

The ordered pipeline: start the containers (unless `SKIP_UP=1`) so they begin their slow boot, generate the SBOMs while that happens (unless `SKIP_SBOMS=1`), then run the uploader, which waits for readiness on its own. The two SKIP switches let you re-run just part of the flow.

A.6 combined_report.py

```

from config import COMPANIES
api_keys = json.load(open(KEYS_FILE))

```

It reuses the same `config.py` and the `api_keys.json` that the uploader produced — that file is the bridge between the two scripts.

```

def api_get(url, api_key):
    req = urllib.request.Request(url, headers={"X-API-Key": api_key})
    try:
        with urllib.request.urlopen(req, timeout=30) as resp:
            if resp.status == 200:
                return json.loads(resp.read().decode("utf-8"))
    except Exception:
        pass
    return None

```

A read-only helper that authenticates with the team API key and returns parsed JSON, or None if anything at all goes wrong. Returning None on failure means one missing metric never crashes the whole report.

```

for cid, company in COMPANIES.items():
    projects = api_get(f"{api}/api/v1/project?pageSize=500&pageNumber=1", key)
    for p in projects:
        m = api_get(f"{api}/api/v1/metrics/project/{p['uuid']}/current", key)
        findings = api_get(f"{api}/api/v1/finding/project/{p['uuid']}", key)
        findings.sort(key=severity_rank)    # criticals first
        top = findings[:TOP_N]
        # assemble: components, per-severity counts, risk score, top vulns

```

For each company it lists the projects, and for each project it asks the API two things: the current metrics (component count, vulnerabilities by severity, risk score) and the individual findings. It sorts the findings so the most severe come first, keeps the top few, and assembles a record matching the report's schema.

`render_html()` (not shown in full) turns that data into one self-contained HTML page: it sums the portfolio totals for the summary cards, then writes a section per company and a card per project with severity badges and a small findings table. All the CSS is inlined, so the file opens anywhere with no external dependencies.

```
json.dump(report, open(json_path, "w"), indent=2)
open(html_path, "w", encoding="utf-8").write(render_html(report))
if total_components == 0:
    print("metrics still computing - wait a few minutes and re-run")
if "--serve" in sys.argv:
    # http.server bound to 0.0.0.0:8090, serving the reports/ folder
```

Finally it writes both the JSON and the HTML, warns you if every metric is still zero (meaning Dependency-Track has not finished analysing yet), and — when called with `--serve` — starts a tiny built-in web server on all interfaces at port 8090 so you can open the dashboard from your host browser.

A.7 How the combined HTML report is built (top to bottom)

An important clarification first: this combined dashboard is entirely our own — it is **not** a Dependency-Track feature or export. Dependency-Track gives us only raw data through its REST API, and it has its own per-instance web interface, but because the three companies live in three separate instances no single Dependency-Track page can show them together. The `render_html()` function builds that page by hand. Here is how, from top to bottom.

1. Roll up the totals. `render_html` receives the list of company records that `collect()` gathered, and first sums them into portfolio-wide figures — how many companies and projects, total components and vulnerabilities, and the count at each severity across the whole estate. These feed the summary cards at the very top of the page.

```
companies = len(report)
projects  = sum(len(c["projects"]) for c in report)
comps     = sum(p["total_components"] for c in report for p in c["projects"])
vulns     = sum(p["total_vulnerabilities"] for c in report for p in c["projects"])
sev_tot   = { ... summed CRITICAL / HIGH / MEDIUM / LOW over every project ... }
```

2. Two small visual helpers. `badge()` emits a coloured pill for a severity and its count (its CSS class, such as 'sev-badge high', gives it the colour); `risk_class()` maps a numeric risk score to a colour — red at 50 or above, orange from 10, green below. This colour-coding is what makes the page quick to scan.

```
def badge(sev, n):
    return f'<span class="sev-badge {sev.lower()}">{sev[:4]} {n}</span>'

def risk_class(r):
    return "risk-high" if r >= 50 else "risk-medium" if r >= 10 else "risk-low"
```

3. Build a card per project. For each project it assembles a card: the name and version, the description, three metric rows (components, vulnerabilities, and the colour-coded risk score), a row of severity badges, and a small table of the top vulnerabilities. Every value taken from Dependency-Track is passed through `html.escape()`, so a stray angle bracket or ampersand in a vulnerability description cannot break the layout or inject markup; a project with no findings shows a green 'No vulnerabilities' line instead.

```
<div class="project-card">                                <!-- shape simplified -->
  <h3>{name} <small>{version}</small></h3>
  <p class="project-desc">{description}</p>
  <div class="metric-row">Components      {components}</div>
  <div class="metric-row">Vulnerabilities {vulnerabilities}</div>
  <div class="metric-row">Risk score      {risk_score}</div>
  <div class="sev-badges">{critical}{high}{medium}{low}</div>
  <table class="vuln-table"> ... top vulnerabilities ... </table>
</div>
```

4. Wrap each company's cards in a section. A company-section has a header with the company name and a clickable link straight to that company's own Dependency-Track UI, and the cards sit in a project-grid — a responsive CSS grid that reflows the cards to fit the window width.

```
<div class="company-section">
  <div class="company-header">
    <span class="company-name">{company_name}</span>
    <a class="company-badge" href="{frontend_url}">{frontend_url}</a>
  </div>
  <div class="project-grid"> ... this company's project cards ... </div>
</div>
```

5. Stitch the whole page together. One large template string returns the finished document, top to bottom: the head with all the CSS inlined in a single style block, the title and a generated timestamp, the row of summary cards built from the totals, then every company section, then a footer. Because the styling is embedded rather than linked to external files, the resulting `.html` is completely self-contained — it opens in any browser on its own and can be emailed or archived as a single file. There is no JavaScript and no front-end framework; the entire page is plain HTML produced by Python string formatting.

```
return f"""<!DOCTYPE html><html><head>
<style> /* all CSS inlined here - dark theme, no external files */ </style>
</head><body>
  <h1>Combined Dependency-Track Report</h1>
  <p class="subtitle"> ... generated {timestamp} ... </p>
  <div class="summary-grid"> 6 cards: Companies, Projects, Components,
                             Vulnerabilities, Critical, High </div>
  {all company sections}
  <div class="footer"> ... </div>
</body></html>"""
```

6. Write it out (and optionally serve it). `main()` saves this string to `reports/combined_report_latest.html` and the same data to `combined_report_latest.json`, so the exact figures are also available in machine-readable form. With `--serve` it then hosts that folder on port 8090 for your browser. In short: Dependency-Track does the analysis and answers questions over its API; this function turns those answers into the single hand-built page you actually look at.